

임의의 weight에 대응하는 MPPI Score 학습

유한 cost 합에서 적분형 cost로의 확장

Volterra 전개 기반 정식화 · 학습 · 추론 · 예제

2026년 9월 18일

요약

기존 접근은 유한개의 cost $C_w = \sum_i w_i c_i$ 를 가정하고, basis vector e_i 에서의 score를 학습한 뒤 역행렬로 임의의 w 에 대한 score를 조립했다. 이 방식은 두 가지 한계를 갖는다. 첫째, cost 개수 n 이 바뀌면 basis 행렬 자체가 정의되지 않는다. 둘째, 선형결합은 근사이며 그 오차는 정량화되지 않은 채로 남는다.

본 문서는 컨셉(특정 weight에서 score를 학습해 임의의 weight로 결합하고 score ascent로 U 를 생성)을 유지한 채, weight를 유한 벡터가 아닌 측도(measure)로 일반화한다. 이때 score는 weight에 대한 Volterra 급수로 전개되며, 기존 선형결합은 이 전개의 1차 절단에 정확히 대응한다. 2차 커널을 추가하면 누락된 공분산 항이 복원되고, 유한 cost는 weight가 Dirac 델타의 합인 특수 경우로 자동 포함된다.

부수적으로 (i) 역행렬이 불필요해지고, (ii) Stein 항등식을 쓰면 rollout 배치 하나에서 무수히 많은 w 에 대한 정답 score를 공짜로 얻으며, (iii) 전개의 유효 범위가 유효 샘플 수(ESS)라는 측정 가능한 지표로 진단된다.

Contents

요약	1
1 배경: MPPI 업데이트는 smoothed Gibbs score이다	2
2 기존 방식과 그 한계	2
3 제안 1: weight를 벡터가 아니라 측도로 본다	3
4 제안 2: score를 w 의 Volterra 급수로 전개한다	3
4.1 기존 방법과의 관계	3
4.2 K_2 의 저계수 분해	3
5 네트워크 구조	4
6 학습 데이터: Stein 항등식으로 배치 하나에서 전부 뽑기	4
6.1 샘플링 분포 설계	5
7 Loss	5
8 추론: score ascent	5
9 유효 조건: 언제 2차 절단으로 충분한가	5
10 구현 순서	6
A 해석적으로 끝까지 풀리는 1차원 예제	7
A.1 설정	7
A.2 참 score (단한 형태)	7
A.3 전개와 절단	7
A.4 일반 공식과의 교차검증	7

A.5 숫자로 보기	8
A.6 정직한 한계	8
B 방향이 왜곡되는 2차원 예제	10
B.1 설정	10
B.2 참 score	10
B.3 숫자	10
C 전체 파이프라인이 도는 모습 (막대 로봇)	11
C.1 문제	11
C.2 학습 (텐서 모양 포함)	11
C.3 네트워크 forward	11
C.4 추론	12
C.5 이 예제에서 확인할 수 있는 것	12

1 배경: MPPI 업데이트는 smoothed Gibbs score이다

제어 시퀀스를 $U \in \mathbb{R}^d$, 비용을 $C_w(U)$ 라 하자. MPPI는 노이즈 $\epsilon \sim \mathcal{N}(0, \Sigma)$ 를 뿌려 다음을 계산한다.

$$J_w(U) = \mathbb{E}_\epsilon \left[e^{-C_w(U+\epsilon)/\lambda} \right], \quad G[w](U) := \nabla_U \log J_w(U), \quad \Delta U = \Sigma G[w](U). \quad (1)$$

여기서 λ 는 온도(temperature)이다. λ 가 작으면 최저비용 샘플 하나에 가중치가 몰리고, 크면 샘플들에 고르게 퍼진다.

용어. $\log J_w$ 는 확률변수 $C_w(U + \epsilon)$ 의 **누적생성함수(cumulant generating function, CGF)**이다. CGF를 전개하면 평균, 분산, 3차 누적량 순으로 항이 나온다.

$$\log J_w = -\frac{1}{\lambda} \kappa_1 + \frac{1}{2\lambda^2} \kappa_2 - \frac{1}{6\lambda^3} \kappa_3 + \dots$$

$\kappa_1 = \mathbb{E}[C_w]$, $\kappa_2 = \text{Var}(C_w)$ 이다. 이 구조가 이후 모든 논의의 출발점이다.

2 기존 방식과 그 한계

기존 정식화는 $C_w = \sum_{i=1}^n w_i c_i$ 를 가정하고 $s_{\theta,i} := G[e_i]$ 를 학습한 뒤, 임의의 $\hat{w}$ 에 대해 $G[\hat{w}] \approx \sum_i \hat{w}_i s_{\theta,i}$ 로 조립한다.

문제는 C_w 가 w 에 선형이어도 $\log J_w$ 는 그렇지 않다는 데 있다. 로그와 적분의 순서를 바꿀 수 없기 때문이다.

$$\log J_{\sum_i w_i c_i} \neq \sum_i w_i \log J_{c_i}. \quad (2)$$

κ_2 항을 펼쳐 보면 누락되는 양이 드러난다.

$$\underbrace{\frac{1}{2\lambda^2} \text{Var}\left(\sum_i w_i c_i\right)}_{\text{참값}} - \underbrace{\frac{1}{2\lambda^2} \sum_i w_i \text{Var}(c_i)}_{\text{선형결합이 주는 값}} = \frac{1}{2\lambda^2} \left[\sum_i (w_i^2 - w_i) \text{Var}(c_i) + 2 \sum_{i < j} w_i w_j \text{Cov}(c_i, c_j) \right]. \quad (3)$$

두 가지 나쁜 소식.

1. 누락항은 cost 항들의 **공분산**에 비례한다. 그런데 로봇 문제에서 cost들은 강하게 상관된다 — 벽 근처에서는 장애물, 관절 리미트, 속도 제한이 동시에 활성화된다. 즉 오차가 0인 경우가 예외이고, 큰 경우가 기본이다.
2. 교차항의 개수가 $O(n^2)$ 이다. n 을 키워서 RL 대비 우위를 확보하려는 전략과 정면으로 충돌한다.

여기에 더해, n 이 런타임에 바뀌면 basis 행렬 $B \in \mathbb{R}^{n \times n}$ 자체가 다른 물건이 되어 역행렬을 정의할 수 없다.

3 제안 1: weight를 벡터가 아니라 속도로 본다

비용을 연속 인덱스 $s \in \mathcal{T}$ 에 대한 적분으로 쓴다.

$$C_w(U) = \int_{\mathcal{T}} w(s) c(U, s) ds. \quad (4)$$

s 는 시간일 수도 있지만, 더 일반적으로는 **제약을 인덱싱하는 연속 파라미터**이다. 예를 들어 로봇 표면 위의 좌표(전신 충돌 회피), 불확실 파라미터(로버스트 제어), 주파수(승차감 H_∞ 가중) 등이다.

핵심 관찰. 유한 cost는 이 형식의 특수 경우이다.

$$w = \sum_i w_i \delta_{s_i} \implies C_w(U) = \sum_i w_i c(U, s_i).$$

Dirac 델타 δ_{s_0} 는 “한 점에만 몰린 무게”로, $\int \delta_{s_0}(s) f(s) ds = f(s_0)$ 를 만족한다.

따라서 “weight가 변한다”와 “cost 개수가 변한다”가 하나의 문제로 통합된다. 전자는 w 의 크기 변화, 후자는 델타의 개수 변화일 뿐이다. 그리고 basis를 델타로 두면 $B = I$ 가 되어 **역행렬이 애초에 필요 없다**.

4 제안 2: score를 w 의 Volterra 급수로 전개한다

C_w 가 w 에 선형이므로 누적량들이 w 에 대해 다중선형 형태로 전개된다.

$$\kappa_1 = \int w(s) \mathbb{E}_\epsilon[c(s)] ds \quad (w\text{에 1차}) \quad (5)$$

$$\kappa_2 = \iint w(s)w(s') \text{Cov}_\epsilon(c(s), c(s')) ds ds' \quad (w\text{에 2차}) \quad (6)$$

즉 $\log J_w$ 는 w 에 대한 **Volterra 급수**(함수를 입력으로 받는 함수의 테일러 전개에 해당)이다. U 로 미분하면 다음을 얻는다.

$$G[w](U) = \underbrace{\int w(s) K_1(U; s) ds}_{1\text{차: 기존 선형결합}} + \underbrace{\iint w(s)w(s') K_2(U; s, s') ds ds'}_{2\text{차: 누락되었던 항}} + O(w^3) \quad (7)$$

$$K_1(U; s) = -\frac{1}{\lambda} \nabla_U \mathbb{E}_\epsilon[c(U + \epsilon, s)], \quad (8)$$

$$K_2(U; s, s') = \frac{1}{2\lambda^2} \nabla_U \text{Cov}_\epsilon(c(U + \epsilon, s), c(U + \epsilon, s')). \quad (9)$$

4.1 기존 방법과의 관계

$w = \delta_{s_0}$ 를 대입하면 $G \approx K_1(U; s_0)$ 이다. 즉 K_1 이 바로 “델타 weight에서의 score”이고, 현재 쓰고 있는 “basis에서 학습 후 선형결합”은 식 (7)의 1차 절단과 정확히 같다.

$$\begin{array}{l} \text{현재 방법} \quad \sum_i w_i K_1(U; s_i) \\ \text{누락된 것} \quad \sum_{i,j} w_i w_j K_2(U; s_i, s_j) \end{array}$$

따라서 “왜 부정확했는가”에 대한 답과 “어떻게 고칠 것인가”에 대한 답이 같은 수식에서 나온다. 2차 커널을 학습 대상에 추가하면 된다.

4.2 K_2 의 저계수 분해

K_2 는 두 연속변수의 함수라 그대로 두면 $O(M^2)$ 비용이 든다. 저계수(low-rank) 분해를 강제한다.

$$K_2(U; s, s') = \sum_{r=1}^R v_r(U) a_r(U; s) a_r(U; s'), \quad v_r \in \mathbb{R}^d, \quad a_r \in \mathbb{R}. \quad (10)$$

그러면 이중적분이 다음으로 축약된다.

$$\iint ww' K_2 = \sum_{r=1}^R v_r(U) \langle w, a_r \rangle^2, \quad \langle w, a_r \rangle = \int w(s) a_r(U; s) ds. \quad (11)$$

비용이 $O(M^2) \rightarrow O(RM)$ 으로 떨어지고, 대칭성 $K_2(s, s') = K_2(s', s)$ 도 자동으로 만족된다. 실용적으로 $R = 8 \sim 16$ 이면 충분하다.

5 네트워크 구조

s 에 의존하는 부분과 그렇지 않은 부분을 분리하는 것이 핵심이다.

```
trunk : h = Enc(x0, U, sigma) # independent of s, 0(1)
index : z(s) = MLP(Fourier(s)) # per-s, very light
head1 : K1(s) = Head1(h, z(s)) -> R^d # 1st-order kernel
head2 : a_r(s) = Head2(h, z(s)) -> R^R # 2nd-order: s-dependent part (R scalars)
head3 : v_r = Head3(h) -> R^{RxD} # 2nd-order: s-independent part
```

- **Fourier feature** $\gamma(s) = [s, \sin 2\pi s, \cos 2\pi s, \sin 4\pi s, \dots]$ 를 쓰는 이유: MLP에 스칼라 s 를 그대로 넣으면 s 에 대해 완만하게 변하는 함수밖에 표현하지 못한다. $w(s)$ 가 좁은 피크를 가질 수 있으므로 K_1 도 s 에 대해 세밀해야 한다.
- 2차 커널의 s -의존 부분이 스칼라 R 개뿐이므로, 2차 항을 추가해도 추론 비용이 거의 늘지 않는다.
- K 개의 제어 샘플을 배치로 넣으면 trunk는 $O(K)$, head는 $O(KM)$ 이다.

6 학습 데이터: Stein 항등식으로 배치 하나에서 전부 뽑기

Stein 항등식은 가우시안 평활화된 함수의 기울기를 미분 없이 구하게 해준다.

$$\nabla_U \mathbb{E}_{\epsilon \sim \mathcal{N}(0, \Sigma)} [f(U + \epsilon)] = \Sigma^{-1} \mathbb{E}_{\epsilon} [\epsilon f(U + \epsilon)]. \quad (12)$$

이를 쓰면 하나의 노이즈 배치에서 K_1 , K_2 , 그리고 임의의 w 에 대한 정답 score를 모두 뽑을 수 있다.

```
1. sample x0, U, sigma
2. eps_k ~ N(0, sigma^2 * Sigma), k = 1..K (K = 1024)
3. rollout -> cost matrix c[k,m] = c(U + eps_k, s_m), m = 1..M
   # <-- the ONLY expensive step
4. MC estimate of kernels (all from the SAME batch):
   K1[m] = -(1/lam) * Sinv @ mean_k( eps_k * c[k,m] )
   K2[m,m'] = (1/2lam^2) * Sinv @ { mean_k( eps_k*c[k,m]*c[k,m'] )
   - mean_k( eps_k*c[k,m] )*mean_k( c[k,m'] )
   - mean_k( c[k,m] )*mean_k( eps_k*c[k,m'] ) }
5. ground-truth score for ANY w (no extra rollout):
   C_w[k] = sum_m beta_m * w(s_m) * c[k,m] # matrix-vector product
   omega = softmax( -C_w / lam )
   G*[w] = Sinv @ sum_k omega_k * eps_k
```

여기서 두 가지가 결정적이다.

(1) **라벨 분산이 줄어든다.** MPPI의 원래 업데이트는 $e^{-C/\lambda}$ 로 가중된 평균이라 유효 샘플 수가 적어 분산이 크다. 반면 K_1 추정에는 $\epsilon_k c_{k,m}$ 의 단순평균이므로 **모든 샘플이 기여한다**. 같은 배치에서 훨씬 정확한 라벨이 나온다.

(2) **배치 하나가 모든 w 를 커버한다.** 5번 단계는 rollout이 필요 없는 행렬 연산이다. 즉 rollout 한 번당 랜덤 w 를 100개 뽑아 정답 score 100개를 얻을 수 있다. **병목이 rollout인데, rollout 하나가 w 하나가 아니라 w 전부를 담당하게 된다.** 이것이 “데이터 수집이 느리다”에 대한 직접적인 답이다.

6.1 샘플링 분포 설계

- x_0 : 작업공간 균등 + 30%는 제약 마진 근처. 그렇지 않으면 $c \approx 0$ 인 데이터만 모인다.
- U : (a) 부드러운 랜덤 노이즈, (b) 랜덤 w 로 돌린 MPPI 해, (c) (b)에 여러 크기 σ 의 노이즈를 더한 것. (c)가 OOD 대비의 핵심이다.
- w : 매끄러운 것(RBF 랜덤 계수) + 좁은 피크형(델타 근사, 유한 cost 케이스) + 부분적으로 0인 것(제약을 끄는 경우, 그리고 위반 영역 커버)을 반드시 섞는다.
- σ : 여러 노이즈 레벨을 조건 입력으로 넣는다(denoising score matching과 같은 구조). 추론 시 annealing이 가능해지고, 다중 최적점에서의 국소 함정 탈출이 덤으로 따라온다.

7 Loss

커널을 따로 회귀하는 것보다, **조립된 score가 맞도록** 학습하는 편이 낫다 (실제로 쓰는 양이 그것이기 때문이다).

$$\mathcal{L} = \mathbb{E}_{(x_0, U, \sigma)} \mathbb{E}_{w \sim \mathcal{W}} \left\| \underbrace{\sum_m \beta_m w_m K_1^\theta(s_m) + \sum_r v_r \langle w, a_r \rangle^2}_{G^\theta[w](U)} - G^*[w](U) \right\|_\Sigma^2 \quad (13)$$

여기서 β_m 은 구적 가중치(적분 $\int f ds \approx \sum_m \beta_m f(s_m)$ 에서 각 점이 담당하는 폭), G^* 는 앞 절에서 공짜로 얻은 MC 정답이다.

보조 손실 두 개를 더하면 학습이 안정된다.

$$\mathcal{L}_1 = \|K_1^\theta(s_m) - \hat{K}_1[m]\|^2 \quad \text{1차 커널을 직접 고정. 없으면 1차/2차 간 책임 분배가 흔들린다.} \quad (14)$$

$$\mathcal{L}_{\text{sm}} = \|\partial_s K_1^\theta\|^2 \quad s \text{에 대한 매끄러움. 학습하지 않은 } s \text{에서의 보간이 좋아진다.} \quad (15)$$

8 추론: score ascent

```
input: x0, weight function w(.), initial U_bar

for sigma in [sigma_max, ..., sigma_min]: # annealing (optional)
  for iter in 1..T:
    1. quadrature s_1..s_M, beta_1..beta_M # denser near peaks of w
    2. K1[m] = K1_theta(U_bar, sigma, s_m) # batched forward
       a[m,r] = Head2(...), v[r] = Head3(...)
    3. assemble: # <-- the ONLY place w appears
       g = sum_m beta_m * w(s_m) * K1[m]
          + sum_r v[r] * ( sum_m beta_m * w(s_m) * a[m,r] )^2
    4. ascent: U_bar <- U_bar + eta * Sigma @ g
```

이 구조에서 따라오는 두 가지 이점.

(1) w 를 바꿔도 2단계는 재사용된다. 따라서 penalty continuation (w 를 약하게 시작해 점진적으로 강하게)이 거의 공짜다. 표준 MPPI는 weight가 바뀔 때마다 K 개 rollout을 전부 다시 해야 한다. stiff한 제약에서 vanilla MPPI를 직접 이기는 지점이 여기서다.

(2) M 을 런타임에 조절할 수 있다. K_1 이 s 의 연속함수이므로 어디서든 평가된다. 여유가 있으면 128점, 급하면 16점. w 가 좁은 피크를 가지면 그 근처만 조밀하게 뽑으면 된다. α -conditioned RL은 학습 시 정한 이산화에 영구히 묶인다.

9 유효 조건: 언제 2차 절단으로 충분한가

전개의 수렴 인자는 $\text{Var}(C_w)/\lambda^2$ 이다. 그런데 이 양은 MPPI의 **유효 샘플 수(ESS)**와 직결된다 — 비용 분산이 λ 대비 크면 가중치가 한 샘플에 몰리고, 그 지점이 곧 전개가 깨지는 지점이다.

$$\text{ESS} \approx \frac{1}{\sum_k \omega_k^2} \text{가 충분히 크다} \iff \text{2차 절단이 유효하다.} \quad (16)$$

실용적 대응은 두 가지다.

1. λ 를 적응적으로 조절해 ESS를 목표값(예: $0.3K$) 근처로 유지한다. 절단 오차가 자동으로 관리된다.
2. w 가 stiff해서 ESS가 어쩔 수 없이 낮아지면, **continuation**으로 여러 번에 나누어 접근한다(앞 절에서 보았듯 거의 공짜다).

이는 방어 논리가 아니라 **진단 지표**다. 실험에서 “ESS 대 score 오차” 그래프를 그리면 이 방법이 언제 유효한지를 정직하게 보일 수 있고, 리뷰어가 물을 질문에 미리 답하는 셈이 된다.

10 구현 순서

1. **Oracle 확인.** 학습 없이, 배치에서 직접 $\hat{K}_1, \hat{K}_2$ 를 뽑아 조립한 score로 ascent. 참 MPPI와 비교한다.
2. **절단 차수 ablation.** 1차만 vs. 1+2차. 여러 λ , 여러 w 에서 참 score와의 cosine similarity를 쟀다. **이 그림이 논문 Figure 1이다.** 현재 방법이 왜 부정확했는지와 2차가 얼마나 복구하는지가 한 장에 들어간다.
3. 그 다음에 네트워크 학습. 1차 커널만 $\rightarrow$ 2차 추가 순으로.
4. **일반화 평가.** 학습 때 보지 않은 w (좁은 피크, 부분 0, 스케일 외삽), M 변화, cost 개수 변화.

2번을 반드시 먼저 하라. 여기서 오차 구조가 확인되면 “학습 문제”와 “정식화 문제”가 깨끗이 분리되고, 이후 디버깅이 훨씬 수월해진다.

	현재	제안
basis	유한 벡터 $e_i + \text{역행렬}$	델타 측도 δ_s , 역행렬 불필요
결합	1차 선형결합	Volterra 전개, 2차까지
cost 개수 변화	깨짐	적분의 항 수만 바뀜
라벨	w 마다 MPPI 실행	배치 하나 $\rightarrow w$ 무제한
라벨 분산	softmax 가중평균 (ESS 낮음)	단순평균 (전 샘플 기여)
유효 조건	불명확	ESS로 진단 가능

컨셉은 그대로이고, 바뀐 것은 **basis**를 델타로, **절단**을 2차로, **라벨**을 Stein 추정으로 이 세 가지다.

부록 A. 해석적으로 끝까지 풀리는 1차원 예제

이 절의 목표는 앞의 모든 주장을 **손으로 확인 가능한 숫자**로 보이는 것이다. 비용을 2차식으로 잡으면 가우시안 적분이 정확히 계산되므로, 참 score와 1차/2차 절단을 모두 닫힌 형태로 비교할 수 있다.

A.1 설정

제어를 스칼라 $U \in \mathbb{R}$, 인덱스를 $s \in [0, 1]$ 로 두고

$$c(U, s) = (U - s)^2, \quad C_w(U) = \int_0^1 w(s) (U - s)^2 ds. \quad (17)$$

해석하자면 “ s 번째 목표는 U 가 s 이기를 원하고, $w(s)$ 는 그 목표의 중요도”이다.

$$W_0 = \int w, \quad W_1 = \int w(s) s ds, \quad W_2 = \int w(s) s^2 ds$$

로 두면 $C_w(U) = W_0 U^2 - 2W_1 U + W_2 = W_0(U - m)^2 + \text{const}$, 여기서 $m = W_1/W_0$ 는 **가중평균 목표**다.

A.2 참 score (닫힌 형태)

$\epsilon \sim \mathcal{N}(0, \sigma^2)$ 에 대해 가우시안 적분을 그대로 수행하면 ($\lambda = 1$ 로 둔다)

$$\log J_w(U) = -\frac{W_0 (U - m)^2}{1 + 2\sigma^2 W_0} + \text{const}, \quad \implies \quad G[w](U) = \frac{-2(W_0 U - W_1)}{1 + 2\sigma^2 W_0} \quad (18)$$

여기서 모든 것이 보인다. 분자 ($W_0 U - W_1$)는 w 에 선형이다. 그런데 분모 ($1 + 2\sigma^2 W_0$)는 선형이 아니다.

$$G[w] = \frac{w \text{에 선형}}{1 + w \text{에 선형}}$$

즉 참 score는 w 에 대한 **유리함수**이고, 선형결합은 그 1차 근사일 뿐이다. 분모는 물리적으로 **스텝 감쇠(damping)** 역할을 한다 — 총 weight W_0 가 크면 평활화된 score가 줄어든다. 1차 절단은 이 감쇠를 무시하므로 **스텝을 과대평가하고 발산하기 쉽다**.

A.3 전개와 절단

$x := 2\sigma^2 W_0$ 로 두고 $1/(1+x) = 1 - x + x^2 - \dots$ 를 쓰면

$$G[w] = \underbrace{-2(W_0 U - W_1)}_{1\text{차 (현재 방법)}} + \underbrace{4\sigma^2 W_0 (W_0 U - W_1)}_{2\text{차 (제안)}} - \underbrace{2(W_0 U - W_1)x^2}_{3\text{차}} + \dots \quad (19)$$

1차항은 w 에 1차, 2차항은 w 에 2차임을 확인할 수 있다 — 식 (7)의 구조 그대로다. 그리고 **수렴비가 정확히** $x = 2\sigma^2 W_0/\lambda$ 이다.

A.4 일반 공식과의 교차검증

본문의 커널 정의가 이 예제에서 실제로 맞는지 확인한다.

$$K_1(U; s) = -\frac{1}{\lambda} \nabla_U \mathbb{E}_\epsilon [(U + \epsilon - s)^2] = -\frac{1}{\lambda} \nabla_U [(U - s)^2 + \sigma^2] = -\frac{2}{\lambda} (U - s) \quad \checkmark \quad (20)$$

$$\text{Cov}_\epsilon(c(s), c(s')) = 4(U - s)(U - s')\sigma^2 + 2\sigma^4 \quad (21)$$

$$K_2(U; s, s') = \frac{1}{2\lambda^2} \nabla_U [4(U - s)(U - s')\sigma^2] = \frac{4\sigma^2}{\lambda^2} \left(U - \frac{s+s'}{2} \right) \quad \checkmark \quad (22)$$

그리고 $\iint w(s)w(s')K_2 ds ds' = 4\sigma^2 W_0(W_0 U - W_1)$ 로, 위 전개의 2차항과 정확히 일치한다. **일반 정식화가 검증되었다**.

A.5 숫자로 보기

$\lambda = 1, \sigma^2 = 0.1$, 평가점 $U = 0$ 으로 고정하고 세 가지 weight를 본다.

	weight $w(s)$	W_0	W_1	$x = 2\sigma^2 W_0$
A	1 on $[0, 1]$ (균등)	1	0.5	0.2
B	4 on $[0.5, 1]$ (한쪽 집중)	2	1.5	0.4
C	10 on $[0, 1]$ (stiff)	10	5	2.0

	참값	1차 (현재)	2차 (제안)	3차
A ($x = 0.2$)	0.8333	1.0000 +20.0%	0.8000 -4.0%	0.8400 +0.8%
B ($x = 0.4$)	2.1429	3.0000 +40.0%	1.8000 -16.0%	2.2800 +6.4%
C ($x = 2.0$)	3.3333	10.0000 +200%	-10.0000 -400%	30.0000 +800%

(참값은 몬테카를로 400만 샘플로 검증했다: 케이스 B에서 MC = 2.1419 vs. 해석해 2.1429.)

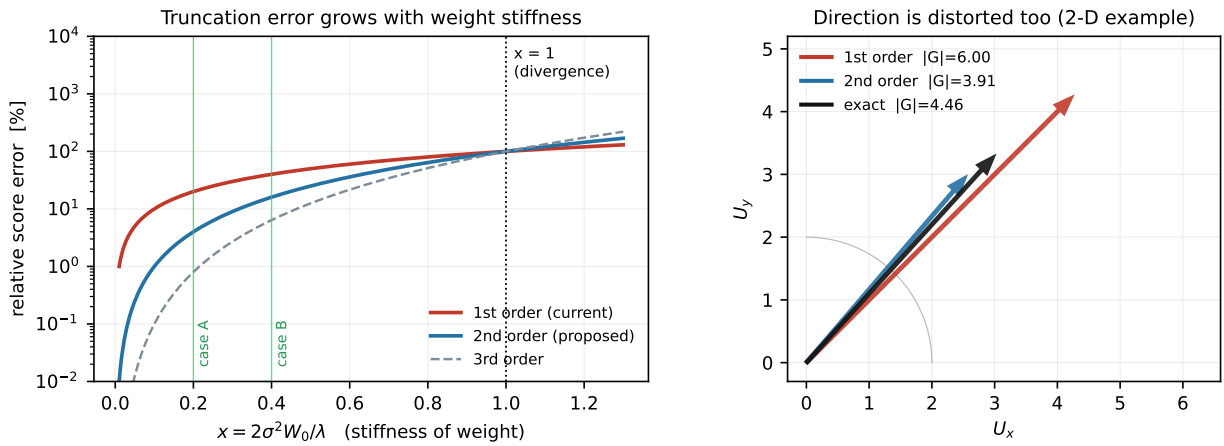


Figure 1: 왼쪽: weight가 stiff해질수록(x 증가) 절단 오차가 커진다. $x < 1$ 에서는 2차 절단이 1차보다 확실히 낮고, $x \geq 1$ 에서는 급수 자체가 발산한다. 오른쪽: 2차원 예제에서 참 score와 각 절단의 방향 비교(부록 B).

읽어야 할 것 세 가지.

1. A, B: 2차가 1차보다 낮다. 오차가 20% → 4%, 40% → 16%로 줄고, 줄어드는 비율이 정확히 x 다.
2. C: stiff하면 급수가 발산한다. 2차를 넣으면 오히려 부호까지 틀린다. 이것은 방법의 실패가 아니라 유효 범위의 경계이며, $x = 2\sigma^2 W_0 / \lambda$ 로 사전에 계산 가능하다.
3. C의 해법은 continuation이다. $w \rightarrow \gamma w$ 로 스케일하면 $x \rightarrow \gamma x$ 가 된다. $\gamma = 0.1$ 부터 시작해 단계적으로 올리면 매 단계가 유효 범위 안에 머문다. 본문에서 보았듯 K_1 은 w 와 무관하므로 이 반복은 재계산 없이 공짜다.

A.6 정직한 한계

이 예제에서 $G = 0 \iff W_0 U = W_1 \iff U = m$ 이며, 이는 1차·2차 절단 모두에서 같다. 즉 2차 비용에서는 절단 오차가 최적점을 옮기지 않고 스텝 크기에만 나타난다.

이는 가우시안 평활화가 2차 함수의 최적점을 이동시키지 않기 때문이다. 최적점 자체가 움직이려면 비용이 비2차여야 한다 — 실제 문제의 hinge형 충돌 비용 $\max(0, \delta - d)^2$ 이나 비볼록 SDF가 그렇다. 다만 그런 경우에는 닫힌 형태가 없어 손계산 예제로 쓸 수 없으므로, 여기서는 구조를 보이는데 집중했다.

한편 스텝 크기 오차도 실제로는 중요하다. 고정된 반복 횟수 안에서 수렴해야 하고, 과대 스텝은 발산을 유발하기 때문이다. 그리고 다차원에서는 **방향까지 왜곡되는데**, 이를 부록 B에서 본다.

부록 B. 방향이 왜곡되는 2차원 예제

B.1 설정

$U = (U_x, U_y) \in \mathbb{R}^2$, 두 개의 cost를 서로 정렬되지 않게 둔다.

$$c_1(U) = U_x^2 \quad (x \text{ 방향 억제}), \quad c_2(U) = \left(\frac{U_x + U_y}{\sqrt{2}} - 1 \right)^2 \quad (\text{대각 방향 목표}). \quad (23)$$

$w_1 = 1, w_2 = 3, \lambda = 1, \sigma^2 = 0.05, \Sigma = I$.

B.2 참 score

$C_w(U) = (U - m)^\top A (U - m) + \text{const}$, $A = w_1 e_x e_x^\top + w_2 n n^\top$, $n = (1, 1)/\sqrt{2}$ 이면 다변량 가우시안 적분에서

$$G[w](U) = -\frac{1}{\lambda} \left(I + \frac{2\sigma^2}{\lambda} A \right)^{-1} \nabla C_w(U) \quad (24)$$

1차원과의 결정적 차이. 감쇠가 스칼라가 아니라 **행렬** $(I + 2\sigma^2 A/\lambda)^{-1}$ 이다. 이 행렬은 A 의 고유방향마다 다른 비율로 축소하므로, 기울기의 크기뿐 아니라 **방향까지 회전시킨다**. 1차 절단은 이 회전을 전혀 반영하지 못한다.

B.3 숫자

$$A = \begin{pmatrix} 2.5 & 1.5 \\ 1.5 & 1.5 \end{pmatrix}, \quad \nabla C_w(0) = -\frac{6}{\sqrt{2}}(1, 1) = (-4.243, -4.243), \quad \rho(2\sigma^2 A) = 0.358.$$

	$G = (G_x, G_y)$	$\ G\ $	방향	참값과의 오차
참값	(2.998, 3.298)	4.457	47.7°	—
1차 (현재)	(4.243, 4.243)	6.000	45.0°	1.562
2차 (제안)	(2.546, 2.970)	3.911	49.4°	0.559

1차 절단은 크기를 35% 과대평가하고 방향도 2.7° 틀린다. 2차를 넣으면 전체 오차가 1.562 → 0.559로 **약 36% 수준으로 줄어든다**.

여기서 방향 오차가 생기는 이유는 두 cost가 서로 직교하지 않기 때문이다 — 즉 $\text{Cov}(c_1, c_2) \neq 0$ 이고, 이것이 바로 식 (3)에서 선형결합이 누락한 바로 그 항이다. 실제 로봇 문제에서 제약들이 강하게 상관된다는 점을 떠올리면, 이 예제가 예외가 아니라 전형임을 알 수 있다.

부록 C. 전체 파이프라인이 도는 모습 (막대 로봇)

앞의 예제는 손계산을 위한 것이었다. 여기서는 실제 구현에 가까운 형태를 본다.

C.1 문제

평면 위를 떠다니는 길이 $L = 0.6\text{ m}$ 막대 로봇. 상태와 제어는

$$x = (p_x, p_y, \theta) \in \mathbb{R}^3, \quad u = (v_x, v_y, \omega) \in \mathbb{R}^3, \quad x_{t+1} = x_t + \Delta t u_t, \quad \Delta t = 0.1, \quad H = 20$$

이고 제어 시퀀스는 $U \in \mathbb{R}^{60}$ 이다. 인덱스 $s \in [0, 1]$ 은 **막대 위의 위치**로, $s = 0$ 과 $s = 1$ 이 양끝이다.

$$q(s; x) = \left(p_x + (s - \frac{1}{2})L \cos \theta, \quad p_y + (s - \frac{1}{2})L \sin \theta \right)$$

비용은 장애물 SDF $d(q) = \min_j (\|q - o_j\| - r_j)$ 와 마진 $\delta = 0.1$ 에 대해

$$c(U, s) = \sum_{t=1}^H \max(0, \delta - d(q(s; x_t)))^2. \quad (25)$$

$w(s)$ 의 물리적 의미. 막대의 각 지점마다 보호 우선순위가 다르다. 끝에 카메라가 달려 있으면 그 부분은 절대 부딪히면 안 되고, 몸통 중간은 조금 긁혀도 된다.

$$w(s) = 1 + 5 \exp(-50(s-1)^2) \quad : s = 1 \text{ 근처만 6배 보호}$$

그리고 이것은 **런타임에 바뀐다** — 파지물의 종류, 센서 장착 위치, 손상 이력에 따라.

C.2 학습 (텐서 모양 포함)

```
# --- collect ONE batch ---
x0 : (3,) # random init state (30% near obstacle margin)
U : (60,) # random / MPPI solution / MPPI solution + noise
eps : (K=1024, 60) # eps_k ~ N(0, sigma^2 I)
s_m : (M=64,) # quadrature points along the rod
beta : (M,) # 1/64 each

c : (K, M) # <-- the ONLY expensive step: rollout + SDF
      # K trajectories x M body points

# --- labels: Stein identity, all from the SAME c matrix ---
K1_hat : (M, 60) # -(1/lam) * mean_k( eps[k] * c[k,m] ) / sigma^2
K2_hat : (M, M, 60) # if needed; usually regress the low-rank form directly

# --- ground-truth scores: reuse the batch for 100 different w ---
for j in 1..100:
  w_j : (M,) # random w (smooth / peaked / partly zero)
  Cw = c @ (beta * w_j) # (K,) <-- matrix-vector product only
  omega = softmax(-Cw / lam) # (K,)
  G_star = (omega[:,None] * eps).sum(0) / sigma^2 # (60,)
  store: (x0, U, sigma, w_j, G_star)
```

비용 회계. rollout 1회($K \times M$ SDF 평가)에서 학습 샘플 100개가 나온다. 기존 방식은 w 하나당 MPPI를 한 번씩 돌려야 했으므로, 같은 예산에서 w 커버리지가 두 자릿수 배 늘어난다.

C.3 네트워크 forward

```
h = Enc(x0, U, sigma) # (256,) independent of s, computed once
z = MLP(Fourier(s_m)) # (M, 64) per-s, cacheable
K1 = Head1(h, z) # (M, 60)
a = Head2(h, z) # (M, R=8)
v = Head3(h) # (R=8, 60)
```

C.4 추론

```
# w appears ONLY in the 3 lines below; everything else is reused as-is.
ws = beta * w(s_m) # (M,)
g = ws @ K1 # (60,) 1st-order term
g += ((ws @ a)**2) @ v # (60,) 2nd-order term -> 0(RM), low-rank
U_bar += eta * Sigma @ g # score ascent

# --- penalty continuation: NO recomputation of K1, a, v ---
for gamma in [0.1, 0.3, 1.0, 3.0]:
    ws = beta * (gamma * w(s_m)) # only this line changes
    ... repeat the 3 lines above ...
```

C.5 이 예제에서 확인할 수 있는 것

확인 항목	방법
정식화가 맞는가	학습 없이 참 K_1, K_2 (rollout에서 직접)로 조립한 score와 brute-force MPPI를 비교. 일치해야 한다. 불일치하면 조립식 버그.
2차가 실제로 도움이 되는가	여러 $w \cdot \lambda$ 에서 1차 대비 cosine similarity. $x = 2\sigma^2 W_0 / \lambda$ 의 함수로 그린다.
w 일반화	학습 때 보지 않은 w (좁은 피크 2개, 부분 0, 스케일 $\times 10$)에서 부위별 제약 위반량을 본다. “ w 를 올린 곳이 실제로 덜 부딪히는가”가 핵심 주장이므로 스칼라 성능보다 이 프로파일이 설득력 있다.
M 런타임 조절	$M = 4 \rightarrow 128$ 로 바꾸며 성능 곡선. RL baseline은 학습 시 이산화에 묶여 평평하고, 제안 방법은 우상향한다.
cost 개수 변화	장애물 개수를 바꾸는 대신, w 를 델타 n 개의 합으로 두고 n 을 변화시킨다. 적분의 항 수만 바뀌므로 코드 변경이 없어야 한다.
stiff 제약	w 스케일을 키우며 ESS 붕괴 지점을 찾고, continuation 유무를 비교한다. vanilla MPPI를 직접 이기는 구간이다.

가장 먼저 할 일. 위 표의 첫 줄, “정식화가 맞는가”를 학습 전에 확인하라. 참 커널로 조립한 score가 brute-force MPPI와 일치하는 것을 보면, 이후 모든 오차가 **학습 오차**임이 확정되어 디버깅 공간이 절반으로 줄어든다.